

SMART CONTRACT SECURITY AUDIT REPORT

Kekius Maximus (KKM) ERC-20 Token Contract

Auditor: Claude (Anthropic) — AI-Assisted Smart Contract Review

Report Date: June 18, 2026

Contract File: KekiusMaximus.sol

Token Standard: ERC-20 (OpenZeppelin Contracts)

Solidity Pragma: ^0.8.35

Overall Risk Rating: Low — No Critical, High, or Medium severity issues identified

This report was generated with the assistance of Claude, an AI model developed by Anthropic. It is intended as a preliminary, automated-assisted code review and does not replace a full manual audit performed by a licensed smart contract security firm. See the Disclaimer section for full limitations.

1. Executive Summary

Claude (Anthropic) was asked to perform a static security and code-quality review of the KekiusMaximus.sol smart contract, an ERC-20 token implementation built on the OpenZeppelin Contracts library. The contract is intentionally minimal: it mints a fixed total supply of 100,000,000 KKM tokens (18 decimals) to a single initialOwner address at deployment and exposes no further privileged functions.

No Critical, High, or Medium severity vulnerabilities were identified. Because the contract contains no custom arithmetic, no external calls, and no post-deployment administrative functions, its attack surface is very small and it inherits the security properties of the underlying OpenZeppelin ERC-20 implementation. The findings below are limited to one Low-severity item and five Informational observations, primarily concerning version pinning, documentation, and centralization disclosure rather than exploitable defects.

This report should be read together with the Disclaimer in Section 8, which describes the scope and limitations of an AI-assisted review of this kind.

2. Audit Scope & Methodology

2.1 Scope

The review covers only the single Solidity source file provided by the client, reproduced in Appendix A:

- File: KekiusMaximus.sol
- Contract: KekiusMaximus (inherits OpenZeppelin ERC20)
- Compiler pragma: ^0.8.35
- License identifier: MIT

Out of scope: the OpenZeppelin library source code itself (assumed to be a genuine, unmodified import), deployment and migration scripts, front-end or wallet integrations, tokenomics and legal/regulatory considerations, and any off-chain infrastructure.

2.2 Methodology

The review combined the following techniques, consistent with standard smart contract audit practice:

- Manual line-by-line source code reading
- Control-flow and state-change analysis (constructor and inherited functions)
- Checklist-based review against common ERC-20 and general smart contract vulnerability classes (reentrancy, integer overflow/underflow, access control, front-running, denial of service, centralization risk)
- Cross-reference against known OpenZeppelin ERC20 implementation behavior

No dynamic testing (unit tests, fuzzing), no formal verification, and no on-chain or testnet deployment were performed as part of this review.

3. Contract Overview

KekiusMaximus is a standard, fixed-supply ERC-20 token:

- Inherits directly from OpenZeppelin's ERC20.sol, with no overridden functions.
- Defines a single constant, INITIAL_SUPPLY = $100,000,000 \times 10^{18}$ (100 million tokens at 18 decimals).
- The constructor accepts one parameter, initialOwner, sets the token name ("Kekius Maximus") and symbol ("KKM"), and mints the entire INITIAL_SUPPLY to initialOwner via the internal _mint function.
- There is no mint, burn, pause, blacklist, ownership, or upgrade functionality beyond what ERC20.sol itself provides (it provides none of these by default).

Functionally, the contract is equivalent to deploying an unmodified OpenZeppelin ERC20 token and minting the full supply once at construction time.

4. Findings Summary

Severity	Count
Critical	0
High	0
Medium	0
Low	1
Informational	5

No issues were identified that could lead to theft of funds, unauthorized minting, denial of service, or loss of user balances under the contract as written.

5. Detailed Findings

LOW-01 — OpenZeppelin Dependency Version Not Pinned

Severity: **Low**

Field	Detail
Description	The contract imports @openzeppelin/contracts/token/ERC20/ERC20.sol without the project's dependency manifest (e.g., package.json, foundry.toml, or remappings.txt) being included in scope, so the exact OpenZeppelin Contracts version being compiled against cannot be confirmed from this file alone.

Impact	If an unpinned or unexpectedly upgraded version of OpenZeppelin is resolved at build time, the deployed bytecode could differ from what was reviewed, potentially including future breaking changes or unaudited code.
Recommendation	Pin an exact, audited OpenZeppelin Contracts release (e.g., via package.json with a fixed version, a lockfile, and matching remappings) and re-verify the resolved version prior to deployment.

INFO-01 — Unconfirmed / Floating Compiler Pragma

Severity: Informational

Field	Detail
Description	The contract declares pragma solidity ^0.8.35, which uses a caret range and allows compilation with any compatible 0.8.x release at or above 0.8.35.
Impact	A floating pragma means the exact compiler version used for the final, deployed bytecode is not guaranteed to match what was reviewed here, which can complicate reproducibility and verification on block explorers.
Recommendation	Pin the pragma to an exact compiler version (e.g., pragma solidity 0.8.35;) once the target Solidity release has been confirmed to be an official, audited compiler release, and compile/deploy with that exact version.

INFO-02 — Centralized Initial Token Distribution

Severity: Informational

Field	Detail
Description	The entire fixed supply (100,000,000 KKM) is minted to a single address, initialOwner, at deployment. The contract provides no on-chain vesting, timelock, or multi-signature requirement for this address.
Impact	If initialOwner is a single externally-owned account (EOA), compromise or loss of that key would put the full token supply at risk. Holders also have no on-chain guarantee regarding how or when the deployer will distribute or otherwise use these tokens.
Recommendation	Consider using a multi-signature wallet or a vesting/timelock contract as initialOwner, and publicly document the intended distribution and any lock-up schedule for transparency.

INFO-03 — No Post-Deployment Administrative Functions

Severity: Informational

Field	Detail
Description	The contract does not implement Ownable, mint, burn, pause, or blacklist functionality beyond the base ERC20 standard. After deployment, the total supply and transfer rules are fixed and immutable.
Impact	This is a positive property from a rug-pull and centralization-risk perspective (no party can mint additional tokens or freeze balances after launch), but it also means there is no recourse to pause or remediate the contract if an unrelated issue is discovered later, and no upgrade path.
Recommendation	No code change required. Disclose this immutability clearly to users/investors as a feature (fixed supply, no admin keys), since it affects expectations around governance and incident response.

INFO-04 — Missing NatSpec Documentation

Severity: Informational

Field	Detail
Description	The contract contains no NatSpec (<code>///</code> , <code>/** */</code>) comments describing the contract, constructor parameters, or constants.
Impact	Reduced readability for auditors, integrators, and automated documentation/verification tooling (e.g., Etherscan's NatSpec display). No security impact.
Recommendation	Add NatSpec comments for the contract, the constructor, and <code>INITIAL_SUPPLY</code> to improve transparency and tooling support.

INFO-05 — No Test Suite Provided for Review

Severity: Informational

Field	Detail
Description	Only the contract source file was supplied; no unit tests, integration tests, or test framework configuration (Hardhat/Foundry) were included in the review scope.
Impact	The absence of a reviewed test suite means deployment-time behavior (e.g., correct constructor argument handling, expected balances post-

Recommendation	mint) has not been independently verified through automated tests as part of this engagement.
	Develop and include a test suite covering deployment with valid and invalid initialOwner values, post-deployment balance and totalSupply checks, and standard ERC-20 transfer/approve/transferFrom behavior, and supply it for review alongside the contract.

6. Gas & Code Quality Observations

- INITIAL_SUPPLY is declared as a constant, so its value is inlined at compile time rather than read from storage — this is gas-efficient and follows best practice.
- Using unmodified OpenZeppelin ERC20 as the base implementation is good practice, as it relies on a widely used, independently audited library rather than custom logic.
- Solidity ^0.8.x's built-in overflow/underflow checks make explicit use of SafeMath unnecessary, and the contract correctly does not include it.
- No gas-inefficient patterns (unbounded loops, redundant storage writes, etc.) were observed, as the contract performs only a single mint operation in the constructor.

7. Recommendations

In priority order:

- Pin the Solidity compiler to an exact, confirmed-released version rather than a floating ^0.8.35 range.
- Pin and document the exact OpenZeppelin Contracts version used, ideally with a lockfile committed to the repository.
- Use a multi-signature wallet or vesting/timelock contract as initialOwner instead of a single EOA, and disclose the distribution plan publicly.
- Add NatSpec documentation and supply an automated test suite for independent verification.
- Engage a licensed, human-led smart contract security firm for a full manual audit prior to any mainnet deployment involving significant value, particularly given that this AI-assisted review has the limitations described in Section 8.

8. Disclaimer & Limitations of This Review

This report was produced with the assistance of Claude, a large language model developed by Anthropic, based solely on a static read of the single source file supplied by the requester. It is intended to support, not replace, a comprehensive smart contract security audit.

Specific limitations of this engagement include:

- No dynamic testing, fuzzing, or formal verification was performed.
- No on-chain or testnet deployment was performed or observed.

- The review did not independently verify the OpenZeppelin library source code, build configuration, or deployment scripts, none of which were provided.
- As with any AI-assisted analysis, there is a possibility that subtle issues were missed or that conclusions could differ from those of a human security researcher; AI-generated findings should be independently verified.
- This report does not constitute legal, financial, or investment advice, and does not assess regulatory compliance, tokenomics, or business/economic risk.

Anthropic and Claude are not a licensed smart contract auditing firm. For contracts intended to hold or control significant value, the authors strongly recommend a complementary audit from an established, human-led smart contract security firm before mainnet deployment.

9. Conclusion

The KekiusMaximus (KKM) contract is a minimal, fixed-supply ERC-20 token built on the OpenZeppelin standard library. Its small surface area — a single constructor call with no further privileged functions — leaves little room for the vulnerability classes that typically affect smart contracts. No Critical, High, or Medium severity issues were found during this review. The Low and Informational items identified relate to version pinning, documentation, and disclosure of centralization at launch, rather than exploitable code defects. Addressing the recommendations in Section 7, and obtaining a complementary professional human audit, would further strengthen confidence in the contract ahead of deployment.

Appendix A: Reviewed Source Code

KekiusMaximus.sol, as supplied for review:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.35;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract KekiusMaximus is ERC20 {
    uint256 public constant INITIAL_SUPPLY = 100_000_000 * 10 ** 18;

    constructor(address initialOwner)
        ERC20("Kekius Maximus", "KKM")
    {
        _mint(initialOwner, INITIAL_SUPPLY);
    }
}
```